

1. Recovery Algorithm (200 words)

- Recovery algorithms in distributed systems ensure that after a failure, the system returns to a consistent global state.
- They aim to restore processes, messages, and data to a condition where the system can continue normal execution without violating correctness.
- A consistent recovery state ensures no lost updates, duplicated operations, or inconsistent message transfers.
- Recovery algorithms work closely with *checkpoints*, *logs*, and *message replay* techniques.
- They identify the failure type (process crash, communication failure, site failure, or network partition).
- After identifying failure, the algorithm decides whether to use **backward recovery** (rollback) or **forward recovery** (roll-forward).
- **Backward recovery** uses saved checkpoints and logs to undo operations to a safe state.
- **Forward recovery** tries to continue execution by reconstructing the missing state using redundant data or replicated information.
- They avoid the “domino effect”—where rollback of one process forces multiple dependent rollbacks.
- Most recovery algorithms maintain message logs so that lost messages can be replayed.
- Recovery must ensure that no orphan processes (inconsistent states) exist.
- Global checkpointing combined with message logging helps guarantee system consistency.
- Recovery algorithms also coordinate between processes to maintain causal and temporal ordering.
- Their main goals are correctness, minimal rollback, reduced overhead, and quick system restoration.

2. Checkpoint (200 words)

- Checkpointing is the process of saving the local state of a process at a certain time so that the system can restart from that point after a failure.
- It provides a snapshot of variables, program counters, buffers, and intermediate results.
- Checkpointing avoids recomputation from the beginning after a failure.
- It is essential for rollback recovery, ensuring the system returns to a consistent global state.
- A **global checkpoint** is a collection of local checkpoints from all processes.
- Checkpoints help prevent the domino effect by defining recovery boundaries.
- There are three main types of checkpointing:
 - **Uncoordinated** – each process takes checkpoints independently, leading to more flexibility but risk of inconsistency.
 - **Coordinated** – all processes take checkpoints together, ensuring global consistency.
 - **Communication-induced** – checkpoints are forced based on message communication patterns.
- Checkpointing requires low overhead and minimal performance impact.
- During recovery, the system rolls back to the most recent consistent checkpoint and replays logged messages.
- Checkpoints can be stored in stable storage such as NVRAM or distributed storage.
- Efficient algorithms ensure minimal disk writes and network overhead.
- Checkpoints are especially useful in long-running distributed applications.

3. Forward and Backward Recovery (200 words)

- Recovery techniques in distributed systems are broadly classified as **forward recovery** and **backward recovery**.

- **Backward recovery (rollback recovery)** means restoring the system to a previously saved state using checkpoints.
 - It is useful when the error has corrupted the system state and cannot be corrected in the current state.
 - Backward recovery relies heavily on periodic checkpoints and message logs.
 - The system rolls back to the nearest consistent checkpoint, discarding incorrect computations.
 - It may cause loss of some work but ensures the system returns to a safe execution point.
 - **Forward recovery (roll-forward)** reconstructs the correct system state without discarding the current state.
 - It uses redundancy, replicated data, or error-correction techniques.
 - Forward recovery is ideal for systems where availability is critical, such as real-time systems.
 - It is more complex than rollback recovery because it requires predicting or calculating the correct future state.
 - Forward recovery is used in fault-tolerant systems, RAID architectures, and replicated databases.
 - Backward recovery is easier to implement and more widely used in typical distributed environments.
 - The choice depends on system requirements, cost, and type of applications.
 - Both aim to maintain system consistency and minimize downtime.
-

4. Failure Recovery (200 words)

- Failure recovery refers to restoring a distributed system to normal functioning after different types of failures.
- Common failures include process crashes, site failures, communication failures, and network partitions.

- The recovery mechanism detects the failure, isolates the faulty component, and initiates corrective actions.
- Recovery uses checkpointing, rollback, replication, and logging techniques.
- The main goal is to bring the system to a consistent global state with minimum data loss.
- If a process fails, local state is restored from the last checkpoint.
- Logged messages are replayed to reconstruct lost computations.
- If an entire node fails, recovery may involve migrating processes to another node.
- Replicated data helps enable forward recovery by reconstructing missing states.
- Recovery protocols also ensure no orphan or inconsistent processes remain.
- In large distributed systems, coordinated recovery is essential to avoid the domino effect.
- Distributed commit protocols like 2PC and 3PC help maintain consistency during transactions.
- Failure recovery also requires stable storage to ensure checkpoints remain safe during failures.
- Efficient recovery ensures high availability, reliability, and fault tolerance in distributed systems.

5. Juang–Venkatesan Algorithm (200 words)

- The Juang–Venkatesan algorithm is a **checkpointing algorithm** designed to provide consistent global checkpoints in distributed systems.
- It belongs to the category of **communication-induced checkpointing**.
- The algorithm forces checkpoints based on message dependencies to avoid the domino effect.

- It tracks “message intervals” and ensures that processes take additional checkpoints whenever causal inconsistencies may arise.
- Each process maintains information about its checkpoint intervals.
- When a message is sent, it includes metadata that helps the receiving process decide if a checkpoint is required.
- If receiving a message may lead to an inconsistent global state, the algorithm forces an induced checkpoint.
- This ensures that the set of checkpoints across processes forms a consistent global state.
- The algorithm reduces the number of forced checkpoints compared to earlier methods.
- It avoids the overhead of fully coordinated checkpointing while still preventing inconsistencies.
- It ensures non-blocking behavior, meaning processes do not stop their normal execution while taking checkpoints.
- The algorithm aims to achieve a balance between performance and fault tolerance.
- It is widely studied for efficiently maintaining global consistency in message-passing systems.

6. Andrew File System (AFS) (200 words)

- The Andrew File System (AFS) is a distributed file system developed at Carnegie Mellon University.
- It provides location-transparent access to files across a distributed environment.
- AFS uses a client–server architecture with centralized file management.
- Its main feature is **client-side caching**, where files are cached locally for faster access.
- When a user opens a file, it is copied to the local machine and operations occur on the cached copy.

- Upon closing the file, updates are sent back to the server.
 - AFS supports **whole-file serving**: entire files are transferred instead of blocks.
 - It uses **callbacks** for maintaining cache consistency—servers notify clients when cached data becomes outdated.
 - AFS divides the file system into volumes, making it easy to replicate, migrate, and manage large data sets.
 - Access control lists (ACLs) provide strong security and permission management.
 - Replication increases availability and fault tolerance.
 - AFS reduces server load because clients work on local cached copies.
 - It is highly scalable and used in large campus or enterprise environments.
 - The design aims for simplicity, transparency, and reduced network traffic.
-

7. DFS (Distributed File System) (200 words)

- A Distributed File System (DFS) allows users to access files stored on remote machines as if they are local.
- It provides transparency in naming, location, access, and replication.
- DFS hides the complexity of distributed storage and presents a unified file system.
- It uses client–server or peer-to-peer architectures.
- Naming transparency allows users to access files without knowing their physical location.
- DFS supports file sharing among multiple users across the network.
- Replication improves reliability and increases availability during failures.
- Cache mechanisms are used to improve performance and reduce server load.
- DFS ensures consistency using update propagation techniques such as write-through, lease-based, or client callbacks.

- Common DFS examples include AFS, NFS, HDFS.
 - DFS provides security through authentication, authorization, and encryption.
 - It uses fault tolerance techniques to handle node or network failures.
 - Scalability is achieved by distributing storage and load across multiple servers.
 - DFS aims to provide a transparent, reliable, and efficient file access service in distributed environments.
-

8. Logging (200 words)

- Logging is the process of recording events, message histories, and state changes to enable recovery in distributed systems.
- Logs are stored in stable storage so they survive failures.
- They help reconstruct the system state during rollback or roll-forward recovery.
- Logging works closely with checkpointing.
- There are two main types: **message logging** and **operation logging**.
- **Message logging** stores all messages received by a process so that they can be replayed after failure.
- **Operation logging** stores high-level actions or updates.
- Logging must ensure no orphan processes occur after recovery.
- There are three logging strategies:
 - **Pessimistic logging** – logs messages synchronously to avoid inconsistencies.
 - **Optimistic logging** – logs asynchronously with better performance but more complex recovery.
 - **Causal logging** – logs only messages that affect the system's causal chain.
- Logs ensure minimal loss of work during crash recovery.

- They support transactional consistency in distributed databases.
 - Efficient logging reduces overhead while preserving correctness.
 - Logging is essential for fault tolerance in distributed systems.
-

9. Detection Algorithm (200 words)

- Detection algorithms in distributed systems identify conditions such as deadlock, termination, or failures.
- In failure detection, algorithms determine whether a process or node has crashed.
- They rely on heartbeats, timeouts, and acknowledgements.
- Failure detectors are classified as perfect, strong, weak, or eventually perfect.
- Deadlock detection algorithms track wait-for graphs and detect circular waits.
- Distributed deadlock detection uses edge-chasing, probe-based methods, or global graph construction.
- Detection algorithms ensure that the system avoids indefinite blocking.
- In termination detection, algorithms detect when all processes become idle and all messages are delivered.
- These algorithms must handle asynchronous communication and avoid false detection.
- Detection is essential for maintaining correctness and ensuring efficient resource usage.
- It supports recovery mechanisms by identifying the source and nature of failures.
- Common detection algorithms include Chandy-Misra-Haas, Huang's algorithm, and failure-detector-based protocols.
- They play a crucial role in distributed operating systems, databases, and cloud platforms.

10. Mutual Exclusion (200 words)

- Mutual exclusion ensures that only one process at a time enters a critical section in a distributed system.
- It prevents race conditions, data inconsistency, and corruption.
- Distributed mutual exclusion is challenging because there is no shared memory or global clock.
- Algorithms use message passing for coordination.
- There are three main categories:
 - **Token-based algorithms** – a token circulates and the holder enters the critical section.
 - **Permission-based algorithms** – processes request permission from peers (e.g., Ricart-Agrawala).
 - **Hybrid algorithms** – combine token and quorum-based ideas.
- Requirements include safety (one process at a time), liveness, fairness, and fault tolerance.
- In token-based methods, failure may cause token loss, requiring regeneration procedures.
- Permission-based algorithms rely on timestamp ordering and logical clocks.
- Mutual exclusion must minimize message overhead.
- It also ensures fairness so no process starves.
- It is vital for distributed databases, file systems, and resource allocation.

If you want, I can also compress these into **80-word** versions for revision!